

Agent 平替 v0 —— 结果报告

Elena (PM) | 2026-06-12 | pilot #1: agent 平替 (上游: v0-team-report-zh.md = 跑之前的计划; 本篇 = 跑完之后的结果。所有数字已对 run 数据逐项 audit, 见 §3。)

0. 结论

在公开 agent 基准 (ALE) 的真任务上跑出第一张「质量 × 成本」表。结论: 没有便宜模型在质量上追平 Claude (最好 GLM-4.6 ≈ 76%), 但 GLM-4.6 在「质量」和「成本」两条线上都是最优候选——76% 质量、约 8× 便宜——足以支撑「便宜模型扛日常、Claude 留难题 + 终审」的 hybrid 路由。v0 量化了「替谁、省多少、何时回退」——这正是 BFC gateway「Smart Cost Routing」缺的那层。

1. 结果: 质量 × 成本表

主表 (apples-to-apples: 取 4 个模型都跑完的 14 道公共题, 口径一致可比)

模型	质量 (mean score)	vs Claude 质量	成本 (\$/题)	便宜倍数
Claude sonnet-4.6 (基准)	0.653	100%	\$0.439	1×
GLM-4.6	0.497	76%	\$0.052	8.4×
DeepSeek V3.2	0.433	66%	\$0.162	2.7×
Qwen3-Coder	0.377	58%	\$0.209	2.1×
Kimi K2	— (路由不可用, 见 §2③)	—	—	—

口径说明:

- 质量 = ALE `ale_run` 自动判分的 per-run 得分 (确定性判分为主, 部分题用 LLM-judge), 同一批题、跨模型可比。
- 成本 = 每个 run 的真实 token 数 × 该模型真实 OpenRouter 单价 (cache_read×0.1、cache_create×1.25) 算出; 不是 harness 里按 Claude 计价的估值 (那个对便宜模型会高估 ~7-14×, 见 §3)。
- 全部模型 harness 固定 `claude_code`、只换 `model:`, 干净单变量。
- 为什么用 14 道公共题: 各模型完成数不同 (Claude 18 / DeepSeek 16 / GLM 17 / Qwen 17), 失败题各异; 只在「都跑完」的题上比才公平。全集数字见 §6 附录。

2. 关键发现

① 没有便宜模型追平 Claude 的质量 (最高 GLM ≈ 76%)。真 agentic 任务上 (开终端、跑工具、产出可验证的交付物), 便宜模型明显逊色。所以 v0 不支持「一个便宜模型整体替掉 Claude」, 支持 hybrid。

② GLM-4.6 是唯一两条线都最优的候选：76% 质量（候选里最高）+ 约 8x 便宜。成本优势的杀手锏是 token 效率，不是单价标签：同一道题（pe_screening，两个模型都真跑出了结果），GLM 质量更高（0.944 vs 0.869）、input token 只用 ~126K，DeepSeek 烧了 ~1.76M（13.9x）；折算成本 GLM \$0.09 vs DeepSeek \$0.41。关键：DeepSeek 的 input 单价其实更低（\$0.23 vs GLM \$0.43/M），却因 token 爆炸 per-task 反而贵 4x。所以全集层面 DeepSeek 单价低也只省 2.7x，GLM 省 8x+。「便不便宜」= token 效率 × 单价，不能只看单价。

③ Kimi K2 经 OpenRouter + claude_code 路由不可用（对 gateway 有直接意义）。修掉限流后，kimi 仍持续撞 provider 端 400 invalid request，能跑完的也近 0 分（全集 mean 0.053）。说明：不是每个「便宜模型」都能干净地通过 gateway 的 Anthropic-compatible 端点驱动 agent harness——正是 BFC「Smart Cost Routing」要替客户挡掉的坑（路由前先验证「这个模型 × 这个 harness」真能跑）。

④ 方法学教训（audit 才发现的，值得团队记住）：比模型必须在「都跑完的公共题集」上比。若用「各自完成题的全集」，DeepSeek 会假性领先（66% > GLM 63%）——因为它跳过了两道自己做不出来的难题、被无形加分；在同一批 14 道题上，GLM（76%）才是真领先。报告口径会直接改变结论，这点对对外话术尤其要明确。

⑤ 战略含义：hybrid 路由 + 「先验证能跑、再按性价比路由」。便宜模型（首选 GLM-4.6）扛能扛的，Claude 留难题 + 终审。v0 这张表就是这套路由的「上游分类器」雏形——ALE 方法学做离线分级，BFC 真实流量再在线细化。

3. 可信度：数字怎么来 + audit 过程

- 成本方法已验证：用 token×单价的公式反算 Claude 单题 = \$0.86，与 harness 自报的 total_cost_usd 完全一致 ✓。便宜模型则用各自真实单价套同一公式。
- 不用 harness 自报成本：它对所有模型都按 Claude 价计，对便宜模型高估 7–14x（实测 DeepSeek 13.9x、Qwen 13.4x、GLM 7.3x），故弃用、改自算。
- 逐项 audit 过：表内每个质量分、成本、token 数都回溯到具体 run 重算；audit 中纠正了三处（DeepSeek 假性领先、一处错误的成本对账、若干倍数四舍五入），才定稿本表。
- OpenRouter 实际账单 = \$67.35（含全程 demo / smoke / 验证 / 限流重试等所有 run，不止主表的那批），表内主跑成本只是其中一部分——账单更高是因为踩坑重试多（见 §4）。
- 每个数字可回溯到 repo `run-log.md` 的 run 记录。

4. 工程证据：真跑出来的问题（不是看文档想象的）

- ★ 发现并修复 ALE 自身一个 reliability bug：它的 zone 容量错误分类函数漏判 GCP 最常见的容量错误（YAML 折行 + 错误码子串不匹配），导致 VM 跨 zone 回退从不触发、一个 zone 缺货就整轮失败。已定位根因、本地修复、写单测验证。（详见 repo `findings.md`，是否回报 Han 待团队定。）
 - 新 GCP 项目配额墙：image 600GB pd-ssd/hyperdisk 超过新项目默认配额（SSD/HDB 各 500GB），逐一申请提额解决。
 - 8-vCPU 容量随负载波动（观察到晚间缺货、清晨可用）→ 真任务 sweep 错峰跑。
 - 这些都不影响判分/模型对比（纯基础设施重试层）。
-

5. 局限

- ALE 分布 $\neq$ 我们的真实流量。v0 是公开基准上的「能力分级 + 短名单」，不等于「在 BFC 客户真实 agent 流量上能替」——要等真实使用数据接力。对外只标「公开基准测得」。
 - 样本量小：公共题集 N=14（全集 N=16-18），near-term 本身分数有噪——趋势可信，绝对值待更大 N 收敛。
 - 任务集：Linux（cpu-free-ubuntu）near-term 一档；Windows / GPU / 更大 N 留待扩展。
 - Kimi 数据不作结论（路由不可用）。
-

6. 给团队/Paul 的决策点

1. 锁达标线（替换门槛）—— Paul 已锁定起步线（2026-06-18）：质量 $\geq 70\%$ / 成本 $\leq \frac{1}{2}$ 。

框架：一个便宜模型「达标 / 进短名单」= 质量 $\geq$ Claude 的 X% 且 成本 $\leq$ Claude 的 1/Y，两条线都过才算。

套到本表（公共 14 题）的三档参考：

档位	质量 门槛 X	成本 门槛 1/Y	过线模型	解读
保守（低风险）	$\geq$ 85%	$\leq$ 1/2	(无)	现有便宜模型都不够；需更强模型或 per-task 路由
中（建议默认）	$\geq$ 70%	$\leq$ 1/3	仅 GLM-4.6	GLM 进短名单，扛能扛的、Claude 兜底
激进（高省钱）	$\geq$ 60%	$\leq$ 1/2	GLM-4.6 + DeepSeek	省更多，但回退多、质量波动大
– 为什么默认 70% / 3x：GLM 在这条线上两项都过（76%、8.4x），DeepSeek（66%）/ Qwen（58%）质量不过线——一刀切下来短名单干净 = 只有 GLM，决策最清晰、对外最好讲。				
– 这只是粗筛门槛（选谁进短名单）。真正的 hybrid 路由是按题 / 按 task_type 判——「便宜模型默认跑，单题没过就回退 Claude」，不是看总均值。v0 已存每道题的分数，v0.1 能据此估算「混合路由的实际省钱率」，再把 X/Y 细化成 per-task 策略。				
– Paul 已定（起步值）：质量 $\geq 70\%$ / 成本 $\leq \frac{1}{2}$ —— 质量门槛取「默认档」，成本门槛比默认更松（ $\frac{1}{2}$ 而非 $\frac{1}{3}$ ）。套表结果：短名单 = 仅 GLM（质量是唯一筛子，DeepSeek/Qwen 成本都已 $\leq \frac{1}{2}$ 、只卡质量）。				
– 套表后的混合省钱率（v0.1）：纯换 GLM = 76% 质量 / 省 8.4x；按题路由（难题回退 Claude）= 99% 质量 / 省 ~2x，正落在 $\leq \frac{1}{2}$ 成本线上。完整短名单 + per-task 分档 + 省钱曲线见 v0.1 结果报告。				

2. 范围（本阶段已定）：这一阶段我们不扩 Windows/GPU 题、不加大 N、也不重测 Kimi。v0.1 的重心是把现有这 14 道题已经跑出的结论，变成一套能落地的路由策略，而不是把基准铺得更广。（Kimi 经 OpenRouter 路由不可用这个结论维持不变，不因为不重测而改变。）
3. 成本：整个 v0 实跑 = OpenRouter \$67 + GCP ~\$10-13（估算，账单滞后）≈ \$77-80，均为公司 pilot 基础设施开销，已逐项留账，需报销。

7. 下一步（谁做什么）

立即 · 本周

- **Paul**：达标线已经锁定（质量 $\geq 70\%$ 、成本 $\leq \frac{1}{2}$ ，见 §6.1），这是一个起步值。它的作用是把「哪些便宜模型有资格进入生产路由」这件事定下来，按现有数据算，短名单只有 GLM。等 Abby 把按题路由的真实落点跑出来、或者接上真实流量数据之后，Paul 再判断这条线要收紧还是维持。
- **Elena (PM)**：负责 v0 与 v0.1 的交付——包括本报告、v0.1 的短名单与路由建议，以及 repo 里的全程证据（run-log 的逐 run 记录、findings 的 bug 写报、质量×成本底表）。英文版按团队需要再出。
- **Abby (SDE)**：把这份离线结论变成真正的路由系统。第一步，用 per-task 矩阵底表复跑一遍按题分档和混合省钱曲线，确认数字对得上（脚本逻辑已审计）。第二步，搭出 verify-then-route 的运行时原型——便宜模型先跑，自动判分，没过就回退 Claude；这一层是兜底机制，不依赖达标线，可以马上动手。第三步，起草一个轻量分类器的方案，把用户 prompt 判成任务类型与难度，并带上置信度，置信度低时默认把任务留给 Claude。

接下来 · 真实流量阶段

- **Abby (SDE)**：一是拿各模型的官方计费价格，核对我们自算的成本；二是想办法从 token 或渠道反推任务类型；三是接入线上的成败信号、对齐打点字段（哪一类任务、用了哪个模型 × harness、成功与否、花了多少钱）。这三件事合起来，是把「在公开基准上离线分级」过渡到「在自有流量上在线细化」的关键一跳，也是从 v0 的「能不能替」走向产品级「实时路由」的核心。
- **Han (ALE 作者)**：确认我们用的 per-run 判分口径是否就是 ALE 的标准口径，以及 ALE 私有题库未来对我们的可用范围。

本阶段不做：不扩 Windows/GPU 题、不加大 N、不重测 Kimi（见 DECISIONS）。

附录：全集数字（N 不等，仅供参考，不作为可比结论）

模型	N	全集 mean	注
Claude sonnet-4.6	18	0.664	
DeepSeek V3.2	16	0.442	缺 cp_test、recsys（失败）
GLM-4.6	17	0.419	缺 nhanes（失败）
Qwen3-Coder	17	0.361	缺 bpmn（失败）
Kimi K2	15	0.053	路由不可用，存疑

⚠ 全集 N 不等、缺的题各异，**跨模型不可直接比**（DeepSeek 因跳过难题会假性偏高）。结论一律以 §1 主表（公共 14 题）为准。

数据与全程证据：repo `agent-eval-v0`（run-log.md / findings.md）。本报告为团队同步，英文版可按需生成。

Repo: <https://github.com/yingshill/agent-eval-v0>